Unit 1: Object-Oriented Programming, Regular Expressions, and PyQt

---

1. Object-Oriented Programming (OOP) in Python

1.1 Classes and Instances

Object-Oriented Programming (OOP) is a paradigm where programs are structured around objects representing real-world entities containing data (properties/attributes) and operations (behaviors/methods).

- **Class**: A class serves as a blueprint or template describing the attributes and methods its objects will contain.
- **Object / Instance**: An individual, concrete entity created from a class.
- **`pass` Statement**: Acts as a syntactic placeholder when defining an empty class temporarily.
- **`self` Parameter**: Standard parameter representing the current object instance, allowing Python to identify which specific object's data is being accessed.
- **`__init__()` Method**: The constructor method executed automatically whenever a new object is instantiated to initialize instance attributes.

```
class Student:
    # Constructor method
    def __init__(self, name, register_number, marks):
        self.name = name                    # Instance attribute
        self.register_number = register_number  # Instance attribute
        self.marks = marks                  # Instance attribute

    def display_details(self):
        print("Student Name:", self.name)
        print("Register Number:", self.register_number)
        print("Marks:", self.marks)

    def calculate_result(self):
        if self.marks >= 40:
            print("Result: Pass")
        else:
            print("Result: Fail")

# Creating instances (objects)
student1 = Student("Amit", "BCA101", 78)
student2 = Student("Neha", "BCA102", 35)

student1.display_details()
student1.calculate_result()
```

1.2 Instance Variables vs. Class Variables

- **Instance Variables**: Variables tied to specific object instances, initialized via `self`; their values differ across individual objects.
- **Class Variables**: Variables shared equally across all instances of a class, defined directly inside the class body outside any method.

```
class BankAccount:
    bank_name = "ABC Bank"  # Class variable shared across all accounts

    def __init__(self, account_holder, balance):
        self.account_holder = account_holder  # Instance variable
        self.balance = balance                # Instance variable

    def deposit(self, amount):
        self.balance += amount
        print(amount, "deposited successfully.")

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
            print(amount, "withdrawn successfully.")
        else:
            print("Insufficient balance.")

    def display_balance(self):
        print("Bank:", BankAccount.bank_name)
        print("Account Holder:", self.account_holder)
        print("Available Balance:", self.balance)

account1 = BankAccount("Rahul", 10000)
account1.deposit(2000)
account1.withdraw(3000)
account1.display_balance()
```

1.3 Inheritance

Inheritance allows a child (derived/subclass) class to acquire the attributes and methods of an existing parent (base/superclass), promoting code reusability. The built-in `super()` function calls methods (such as `__init__`) from the parent class.

1. **Single Inheritance**: A single child class inherits from a single parent class.
2. **Multilevel Inheritance**: A derived class inherits from a parent, which in turn inherits from another base class (e.g., `Person` \(\rightarrow\) `Employee` \(\rightarrow\) `Manager`).
3. **Hierarchical Inheritance**: Multiple child classes inherit directly from the same parent base class.
4. **Multiple Inheritance**: A single child class inherits directly from two or more distinct parent classes.

```python
# 1. Single Inheritance & super()
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display_person(self):
        print(f"Name: {self.name}, Age: {self.age}")

class Student(Person):
    def __init__(self, name, age, course):
        super().__init__(name, age)
        self.course = course

    def display_student(self):
        self.display_person()
        print("Course:", self.course)

# 2. Multiple Inheritance
class Camera:
    def take_photo(self):
        print("Photo captured.")

class Phone:
    def make_call(self):
        print("Calling a contact.")

class Smartphone(Camera, Phone):
    def browse_internet(self):
        print("Browsing the internet.")

device = Smartphone()
device.take_photo()
device.make_call()
device.browse_internet()
```

1.4 Polymorphism

Polymorphism means "many forms" and allows the same method interface to exhibit different behaviors depending on the calling object.

- **Method Overriding**: When a child class redefines a method that is already provided by its parent class.
- **Polymorphism via Common Function / Duck Typing**: A shared function invokes a method name across different object types without inspecting their explicit class type.
- **Operator Overloading**: Defining custom behavior for Python operators (such as +) on user-defined objects using special dunder methods (such as `__add__`).

```python
# Method Overriding & Common Function
class CreditCardPayment:
    def pay(self, amount):
        print(f"Paid {amount} using a Credit Card.")

class UPIPayment:
    def pay(self, amount):
        print(f"Paid {amount} using UPI.")

def process_bill(payment_method, amount):
    payment_method.pay(amount)  # Duck typing polymorphism

process_bill(CreditCardPayment(), 2500)
process_bill(UPIPayment(), 1200)

# Operator Overloading
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

    def __add__(self, other):
        return self.price + other.price

p1 = Product("Keyboard", 1500)
p2 = Product("Mouse", 700)
print("Total Price:", p1 + p2)  # Calls p1.__add__(p2)
```

1.5 Abstract Classes

Abstraction hides internal implementation details while exposing a clear, standardized interface. Python supports abstract classes via the built-in **`abc`** module.

- A class inheriting from **`ABC`** that defines at least one method decorated with **`@abstractmethod`** becomes an abstract class.
- **Direct instantiation is disallowed**: You cannot instantiate an abstract class directly; subclasses must provide concrete implementations for all abstract methods.

```python
from abc import ABC, abstractmethod

class Employee(ABC):
    def __init__(self, name):
        self.name = name

    @abstractmethod
    def calculate_salary(self):
        pass

class FullTimeEmployee(Employee):
    def __init__(self, name, monthly_salary):
```

```
        super().__init__(name)
        self.monthly_salary = monthly_salary

    def calculate_salary(self):
        return self.monthly_salary

class PartTimeEmployee(Employee):
    def __init__(self, name, hours_worked, hourly_rate):
        super().__init__(name)
        self.hours_worked = hours_worked
        self.hourly_rate = hourly_rate

    def calculate_salary(self):
        return self.hours_worked * self.hourly_rate

emp1 = FullTimeEmployee("Ravi", 50000)
emp2 = PartTimeEmployee("Sneha", 80, 500)
print(emp1.name, "Salary:", emp1.calculate_salary())
print(emp2.name, "Salary:", emp2.calculate_salary())
```

---

2. Exception Handling in Python

2.1 Concepts and Built-in Exceptions

An **exception** is a runtime error interrupting normal program flow. Unhandled exceptions terminate execution immediately.

Exception Type	Trigger Situation
`ZeroDivisionError`	A number is divided by zero.
`ValueError`	An operation or function receives an argument that has the right type but an inappropriate value.
`TypeError`	An operation is executed on an incompatible data type.
`IndexError`	An invalid sequence index is accessed.
`KeyError`	A non-existent dictionary key is referenced.
`FileNotFoundError`	An attempt is made to open a missing file.

2.2 `try`, `except`, `else`, and `finally`

- **`try`**: Encapsulates code statements that could potentially trigger a runtime exception.
- **`except`**: Captures and handles specific exception types.
- **`else`**: Runs exclusively when the `try` block succeeds without raising any exceptions.
- **`finally`**: Executes unconditionally, regardless of whether exceptions were raised or handled; used for releasing resources and cleanup operations.

```
try:
    marks = int(input("Enter student marks: "))
    if marks < 0 or marks > 100:
        raise ValueError("Marks must be between 0 and 100.")
except ValueError as error:
    print("Invalid input:", error)
else:
    print("Marks entered successfully:", marks)
    result = "Pass" if marks >= 40 else "Fail"
    print("Result:", result)
finally:
    print("Marks processing completed.")
```

2.3 Raising Exceptions & User-Defined Exceptions

The **`raise`** statement intentionally generates exceptions when domain rules are violated. Programmers can create custom user-defined exceptions by subclassing Python's base **`Exception`** class.

```
# Custom User-Defined Exception
class InvalidAgeError(Exception):
    def __init__(self, age, message="Applicant must be at least 18 years old"):
        self.age = age
        self.message = message
        super().__init__(self.message)

def register_voter(age):
    if age < 18:
        raise InvalidAgeError(age)
    print("Voter registered successfully.")

try:
    user_age = int(input("Enter applicant age: "))
    register_voter(user_age)
except InvalidAgeError as error:
    print("Registration failed:", error)
except ValueError:
    print("Age must be entered as a numeric value.")
```

---

3. Regular Expressions Using the `re` Module

3.1 Regex Syntax and Metacharacters

A **regular expression (regex)** is a text pattern utilized to search, match, split, or replace character sequences. In Python, regular expressions are imported via the `re` module and written as raw strings (`r"..."`) to ensure backslashes are not treated as escape characters.

Symbol	Description	Example Pattern
`.`	Any character except a newline.	`a.c`
`^` / `$`	Start of string / End of string.	`^Hello` / `end$`
`*` / `+` / `?`	0 or more / 1 or more / 0 or 1 occurrence.	`ab*` / `ab+` / `colou?r`
`{n}` / `{m,n}`	Exactly \(n\) times / Between \(m\) and \(n\) times.	`\d{10}` / `\d{4,6}`
`\d` / `\D`	Any numeric digit `` / Any non-digit.	`\d{3}`
`\w` / `\W`	Word character (alphanumeric + _) / Non-word.	`\w+`
`\b`	Word boundary.	`\b\d{10}\b`
`(?=...)`	Positive lookahead assertion (checks without consuming).	`(?=.*[A-Z])`

3.2 Functions of the `re` Module

- **`re.fullmatch(pattern, string)`**: Returns a match object if the entire target string conforms to the pattern, else `None`.
- **`re.search(pattern, string)`**: Searches for the first occurrence of the pattern anywhere in the string.
- **`re.findall(pattern, string)`**: Returns all non-overlapping matching occurrences in a list.
- **`re.sub(pattern, replacement, string)`**: Replaces substrings matching the pattern with alternative text.
- **`re.split(pattern, string)`**: Splits text using pattern delimiters.

```
import re

text = "Contact: 9876543210 or 8765432109. Card: 1234-5678-9876-5432."

# 1. Extraction with re.findall
mobile_pattern = r"\b\d{9}\b"
mobiles = re.findall(mobile_pattern, text)
print("Mobile Numbers:", mobiles)

# 2. Masking sensitive data with re.sub
masked_text = re.sub(r"\d{4}-\d{4}-\d{4}-\d{4}", "XXXX-XXXX-XXXX-XXXX", text)
print("Masked String:", masked_text)
```

---

4. GUI Programming Using PyQt

4.1 CLI vs. GUI & PyQt Architecture

- **CLI (Command-Line Interface)** relies on typing commands and string inputs via the console, demanding command familiarity and offering basic usability.
- **GUI (Graphical User Interface)** uses visual elements (windows, text inputs, radio buttons, message boxes) that accept mouse, touch, and keyboard interactions.
- **Qt**: A C++ cross-platform framework for building graphical software.
- **PyQt**: The official Python binding exposing Qt classes and GUI widgets to Python.

Architecture Layers:

1. **Python Application Code**: Business logic written by the developer.
2. **PyQt Binding**: Bridges Python code with Qt's underlying C++ classes.
3. **Qt Framework**: Manages widgets, layout managers, window graphics, and internal event loops.
4. **Operating System**: Manages display hardware, hardware inputs (mouse clicks, keystrokes), and window boundaries.

The communication path is: **User Action \(\rightarrow\) OS \(\rightarrow\) Qt Event Loop \(\rightarrow\) Widget \(\rightarrow\) Python Slot Method**.

4.2 Core PyQt Modules

- **`QtWidgets`**: Houses visual interface components (`QApplication`, `QWidget`, `QLabel`, `QLineEdit`, `QPushButton`, `QComboBox`, `QRadioButton`, `QCheckBox`, `QMessageBox`).
- **`QtCore`**: Houses fundamental non-GUI functionality (core signals and slots, event handling loops, timers, object base `QObject`, constants such as `Qt.Horizontal`).
- **`QtGui`**: Houses graphical support classes (fonts, color configurations, icons, pixmaps, direct paint engines).

4.3 Layout Management

Rather than manually placing widgets using fixed pixel coordinates (`move()` / `setGeometry()`), layout managers automatically resize and reposition child widgets responsively when windows scale.

- **`QVBoxLayout`**: Stacks widgets vertically in a column.
- **`QHBoxLayout`**: Places widgets horizontally in a row.
- **`QGridLayout`**: Positions components inside a 2D matrix of rows and columns.
- **`QFormLayout`**: Organizes two-column label-input pairs for forms.

4.4 Signals, Slots, and Event Handling

- **Signal**: A notification emitted by a widget whenever an interaction or state alteration occurs (e.g., `clicked`, `textChanged`, `returnPressed`).
- **Slot**: A Python function or method connected to respond when the signal fires.
- **Syntax**: `widget.signal.connect(slot_method)` — the method name is passed **without parentheses** so it is referenced as a callback rather than executed immediately.

- **Event Handling**: Events represent low-level actions dispatched by Qt's event system. Windows handle events by overriding built-in methods:
  - `mousePressEvent(event)`: Intercepts mouse button presses and retrieves cursor coordinates via `event.x()` and `event.y()`.
  - `keyPressEvent(event)`: Intercepts keystrokes, checking keys like `Qt.Key_Escape`.
  - `closeEvent(event)`: Intercepts window closing to confirm exit via `event.accept()` or `event.ignore()`.

---

5. Complete Implementations for Unit-1 Lab Exercises

Lab Exercise 1: Demonstrate Object-Oriented Programming Concepts

- **Objective**: Implement classes, constructors, private/protected encapsulation with getters/setters, inheritance with `super()`, abstract classes (`ABC`), and runtime polymorphism.

```
"""
Lab Exercise 1: Demonstration of OOP Concepts in Python
Covers: Abstraction, Encapsulation, Inheritance, Polymorphism, and Constructors
"""
from abc import ABC, abstractmethod

class Person(ABC):
    def __init__(self, name, age):
        self._name = name        # Protected attribute
        self.__age = age         # Private attribute (Encapsulation)

    # Getter for private attribute
    def get_age(self):
        return self.__age

    # Setter with validation
    def set_age(self, age):
        if age > 0:
            self.__age = age
        else:
            print("Age must be positive.")

    # Abstract method enforcing implementation in subclasses
    @abstractmethod
    def display_role(self):
        pass

    def display_info(self):
        print(f"Name: {self._name}, Age: {self.__age}")

class Student(Person):
    def __init__(self, name, age, student_id, course):
        super().__init__(name, age)
        self.student_id = student_id
        self.course = course

    # Overriding abstract method
    def display_role(self):
        print(f"{self._name} is a Student (ID: {self.student_id}, Course: {self.course})")

    # Method overriding extending parent display_info
    def display_info(self):
        super().display_info()
        print(f"Student ID: {self.student_id}, Course: {self.course}")

class Teacher(Person):
    def __init__(self, name, age, subject, experience):
        super().__init__(name, age)
        self.subject = subject
        self.experience = experience

    def display_role(self):
        print(f"{self._name} is a Teacher (Subject: {self.subject}, Experience: {self.experience} years)")

    def display_info(self):
        super().display_info()
        print(f"Subject: {self.subject}, Experience: {self.experience} years")

# Polymorphic dispatcher function
def show_role(person: Person):
    person.display_role()

if __name__ == "__main__":
    print("=" * 55)
    print("  OBJECT-ORIENTED PROGRAMMING CONCEPTS DEMONSTRATION")
    print("=" * 55)

    # Instantiating concrete subclasses
    student = Student("Tanishq", 20, "BCA2024001", "BCA")
    teacher = Teacher("Dr. Sharma", 45, "Python", 15)

    print("\n--- 1. Display Details (Inheritance & Super) ---")
    student.display_info()
    print()
    teacher.display_info()

    print("\n--- 2. Polymorphism Demonstration ---")
    community_members = [student, teacher]
    for member in community_members:
        show_role(member)
```

```
    print("\n--- 3. Encapsulation (Getters and Setters) ---")
    print(f"Original Student Age: {student.get_age()}")
    student.set_age(21)
    print(f"Updated Student Age: {student.get_age()}")
    student.set_age(-5)  # Triggers validation rule
```

Lab Exercise 2: Regular Expression–Based Validation for Login / Registration Forms

- **Objective**: Implement comprehensive regular expression validation checking field constraints for Full Name, Username, Email, 10-digit Indian Mobile Number, and Strong Password (using positive lookaheads).

```
"""
Lab Exercise 2: Regex-based Validation for Registration / Login Form
Covers: re.fullmatch(), metacharacters, character sets, and positive lookaheads
"""
import re

class RegistrationValidator:
    def validate_name(self, name):
        # Only alphabetic letters and spaces, length 2 to 50
        pattern = r"^[A-Za-z ]{2,50}$"
        return bool(re.fullmatch(pattern, name))

    def validate_username(self, username):
        # Starts with an alphabet letter, alphanumeric/underscores, 5-15 characters
        pattern = r"^[A-Za-z][A-Za-z0-9_]{4,14}$"
        return bool(re.fullmatch(pattern, username))

    def validate_email(self, email):
        # RFC standard compliant email structure
        pattern = r"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$"
        return bool(re.fullmatch(pattern, email))

    def validate_mobile(self, mobile):
        # 10 digits starting with 6, 7, 8, or 9
        pattern = r"^\d{9}$"
        return bool(re.fullmatch(pattern, mobile))

    def validate_password(self, password):
        # Min 8 chars: >=1 lowercase, >=1 uppercase, >=1 digit, >=1 special character
        pattern = r"^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@#$%&*!])[A-Za-z\d@#$%&*!]{8,}$"
        return bool(re.fullmatch(pattern, password))

if __name__ == "__main__":
    validator = RegistrationValidator()

    print("=" * 55)
    print("  USER REGISTRATION FORM VALIDATION (REGEX)")
    print("=" * 55)

    name = input("Enter Full Name: ").strip()
    username = input("Enter Username: ").strip()
    email = input("Enter Email Address: ").strip()
    mobile = input("Enter Mobile Number: ").strip()
    password = input("Enter Password: ").strip()

    errors = []

    if not validator.validate_name(name):
        errors.append("Name must contain letters and spaces only (2-50 chars).")
    if not validator.validate_username(username):
        errors.append("Username must begin with a letter and contain 5-15 alphanumeric characters.")
    if not validator.validate_email(email):
        errors.append("Invalid email address format (example: user@domain.com).")
    if not validator.validate_mobile(mobile):
        errors.append("Mobile number must be exactly 10 digits starting with 6, 7, 8, or 9.")
    if not validator.validate_password(password):
        errors.append("Password must be at least 8 characters long and contain uppercase, lowercase, digit, and special character (@#$%&*!).")

    print("\n--- VALIDATION RESULT ---")
    if not errors:
        print("Registration completed successfully! All fields conform to regex rules.")
    else:
        print("Registration failed with the following errors:")
        for err in errors:
            print(f" [!] {err}")
```

Lab Exercise 3: PyQt GUI Application for Input Validation & Dialog Results

- **Objective**: Construct a complete PyQt5 application that takes user credentials (Email and Password), validates them using regular expressions and exception handling, hides the password using `EchoMode.Password`, binds keyboard shortcuts via `keyPressEvent`, and delivers feedback via **`QMessageBox`** dialog boxes.

```
"""
Lab Exercise 3: PyQt GUI Application with Input Validation & Dialog Boxes
Covers: QApplication, QMainWindow, QLineEdit, QPushButton, QMessageBox, and Event Handling
"""
import sys
import re
from PyQt5.QtWidgets import QApplication, QMainWindow, QLabel, QLineEdit, QPushButton, QMessageBox
from PyQt5.QtCore import Qt

class LoginWindow(QMainWindow):
    def __init__(self):
```

```
        super().__init__()
        # Predefined credentials for successful login verification
        self.registered_email = "admin@admin.com"
        self.registered_password = "Admin@147"
        self.init_ui()

    def init_ui(self):
        self.setWindowTitle("Login & Input Validation Application")
        self.setGeometry(600, 250, 500, 320)

        # Email Label and LineEdit
        self.label_email = QLabel("Email Address:", self)
        self.label_email.setGeometry(50, 45, 120, 30)

        self.input_email = QLineEdit(self)
        self.input_email.setPlaceholderText("Enter your email")
        self.input_email.setGeometry(180, 45, 260, 30)

        # Password Label and LineEdit
        self.label_pass = QLabel("Password:", self)
        self.label_pass.setGeometry(50, 95, 120, 30)

        self.input_pass = QLineEdit(self)
        self.input_pass.setPlaceholderText("Enter your password")
        self.input_pass.setEchoMode(QLineEdit.EchoMode.Password)  # Masks characters for security
        self.input_pass.setGeometry(180, 95, 260, 30)

        # Submit / Login Button
        self.btn_login = QPushButton("Login", self)
        self.btn_login.setGeometry(180, 150, 110, 35)
        # Connecting signal to slot
        self.btn_login.clicked.connect(self.check_login)

        # Clear Button
        self.btn_clear = QPushButton("Clear", self)
        self.btn_clear.setGeometry(300, 150, 110, 35)
        self.btn_clear.clicked.connect(self.clear_fields)

        # Status note label
        self.lbl_note = QLabel("Press 'ESC' key to close application.", self)
        self.lbl_note.setGeometry(150, 210, 260, 25)

    # Event Handling: Overriding keyPressEvent for Escape key shortcut
    def keyPressEvent(self, event):
        if event.key() == Qt.Key_Escape:
            self.close()
        else:
            super().keyPressEvent(event)

    def check_login(self):
        entered_email = self.input_email.text().strip()
        entered_pass = self.input_pass.text().strip()

        # Regular expression validation patterns
        email_regex = r"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$"
        password_regex = r"^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$"

        # Input validation using QMessageBox dialog boxes
        if not re.fullmatch(email_regex, entered_email):
            QMessageBox.warning(
                self,
                "Invalid Email",
                "Please enter a valid email format (e.g. name@domain.com)."
            )
            return

        if not re.fullmatch(password_regex, entered_pass):
            QMessageBox.warning(
                self,
                "Weak Password",
                "Password must be at least 8 characters long and contain:\n"
                "• At least 1 uppercase letter\n"
                "• At least 1 lowercase letter\n"
                "• At least 1 number\n"
                "• At least 1 special character (@$!%*?&)"
            )
            return

        # Authentication verification
        if entered_email == self.registered_email and entered_pass == self.registered_password:
            QMessageBox.information(
                self,
                "Authentication Success",
                f"Welcome! Login successful for {entered_email}."
            )
        else:
            QMessageBox.critical(
                self,
                "Authentication Error",
                "Invalid Email or Password. Please check your credentials."
            )

    def clear_fields(self):
        self.input_email.clear()
        self.input_pass.clear()
        self.input_email.setFocus()

if __name__ == "__main__":
```

```
    app = QApplication(sys.argv)
    window = LoginWindow()
    window.show()
    sys.exit(app.exec_())
```

Unit 2: Python APIs for Web Applications

---

1. Application Programming Interfaces (APIs) in Web Applications

1.1 Role of APIs in Web Applications

An **Application Programming Interface (API)** acts as an intermediary interface that enables an application to communicate with a remote server over the web. Instead of embedding all application data directly inside the client code, an API allows applications to fetch, submit, update, or remove data stored on external servers.

For instance, an application like Instagram or a Student Management System relies on an API to communicate user actions (such as viewing, creating, editing, or deleting posts and student records) to the backend server.

1.2 Client–Server Architecture

Modern web applications adhere to a **Client–Server Architecture**:

- **Client (Python Program / Frontend)**: Initiates an action by sending an HTTP request across the network to access or manipulate a resource.
- **Server**: Receives the incoming request, processes business logic, interacts with databases, and returns an HTTP response containing a status code and payload data.

```
Python Program (Client)
        │
        ▼  HTTP Request (e.g., GET /users/1)
JSONPlaceholder / Remote Server
        │
        ▼  HTTP Response (Status Code + JSON Body)
Python Program (Client)
```

(Communication workflow between client and server)

1.3 Basics of HTTP Protocol & The Request–Response Cycle

Communication between client applications and web APIs operates on the **Hypertext Transfer Protocol (HTTP)**.

The Request–Response Cycle:

1. **Request Construction**: The client defines an endpoint URL, selects an HTTP method (such as `GET`), and optionally configures parameters, headers, or a payload.
2. **Transmission & Processing**: The request is transmitted across the internet to the destination server.
3. **Response Delivery**: The server transmits back an HTTP response containing:
    - **Status Code**: A numeric code indicating request outcome (e.g., `200 OK` for success, `404 Not Found` for missing resources, `500` for server errors).
    - **Response Body**: Structured data, typically formatted as JSON.

1.4 RESTful Web Services

REST stands for **Representational State Transfer**. A **RESTful API** is a web API designed around **resources**, which are accessed via standardized URLs using standard HTTP methods.

Core Concepts of REST:

- **Resource**: The fundamental entity or object that the application manages (e.g., `students`, `teachers`, `courses`, `attendance`, `marks`).
- **Endpoint**: A specific Uniform Resource Locator (URL) targeting a distinct resource or operation on the server.
    - Base API Service: `https://api.example.com`
    - Resource Collection Endpoint: `https://api.example.com/students`
    - Specific Item Endpoint: `https://api.example.com/students/101`

Primary HTTP Methods in RESTful Services:

HTTP Method	Purpose / Operation	Example Action (University System)	Example Action (Social / Orders)
`GET`	**Retrieve** existing data	View student information	View user orders / View a post
`POST`	**Create** a new record	Register / add a new student	Create a new post
`PUT`	**Update** an existing record	Update student information	Edit profile / Modify a post
`DELETE`	**Delete** a record	Delete a student record	Delete a post

---

2. Working with URLs and Data Exchange Using JSON

2.1 Working with URLs and Dynamic Endpoints

An API endpoint is constructed by combining the base domain, the resource path, and optional path parameters or query identifiers.

In Python, dynamic endpoints targeting specific records can be constructed using string concatenation or formatted strings:

```
user_id = input("Enter User ID (1-10): ")  # e.g., "5"
# Dynamic endpoint construction
url = "https://jsonplaceholder.typicode.com/users/" + user_id
# Resulting endpoint: https://jsonplaceholder.typicode.com/users/5
```

(Dynamic endpoint generation targeting individual records)

2.2 Data Exchange Using JSON (JavaScript Object Notation)

JSON is a lightweight, human-readable text format widely utilized for exchanging structured data between client applications and web servers.

1. Key–Value Pairs:

JSON organizes data into key-value pairs enclosed in curly braces `{}`. Keys are strings wrapped in double quotes, and values can be strings, numbers, booleans, arrays, or nested objects.

```
{
  "id": 101,
  "name": "Rahul",
  "course": "BCA",
  "marks": 82
}
```

- `"name"` is the key, and `"Rahul"` is the value.
- `"marks"` is the key, and `82` is the numeric value.

2. JSON Arrays:

A JSON array represents an ordered list of elements enclosed in square brackets `[]`:

```
student = {
    "name": "Rahul",
    "skills": ["Python", "Java", "SQL"]
}
# Accessing third element (index 2)
print(student["skills"])  # Output: SQL
```

(Extracting indexed elements from a JSON array)

3. Nested JSON:

API responses frequently nest objects within other objects. Python accesses nested fields by chaining dictionary keys:

```
student = {
    "name": "Rahul",
    "address": {
        "city": "Pune",
        "state": "Maharashtra"
    }
}

print("Student Name:", student["name"])           # Rahul
print("City:", student["address"]["city"])         # Pune
print("State:", student["address"]["state"])       # Maharashtra
```

(Accessing child dictionary keys inside nested JSON structures)

2.3 Mapping JSON to Python Objects

When consuming an API in Python, JSON data maps directly onto native Python data types:

- JSON Objects (`{...}`) \(\rightarrow\) **Python Dictionaries** (`dict`)
- JSON Arrays (`[...]`) \(\rightarrow\) **Python Lists** (`list`)
- JSON Strings, Numbers, Booleans \(\rightarrow\) `str`, `int`/`float`, `bool`

---

3. Consuming Web APIs Using Python

3.1 The `requests` Library

Python offers several modules for network communication, but the third-party **`requests`** library is the standard choice because it simplifies sending HTTP requests and parsing responses.

Installation:

```
pip install requests
```

(Installing the requests package via terminal)

3.2 Basic Request and Response Methods

- **`requests.get(url, timeout=...)`**: Sends an HTTP `GET` request to the targeted endpoint URL.
- **`response.status_code`**: Returns the numeric HTTP status code indicating server status.
- **`response.json()`**: Automatically parses and deserializes the JSON response body into native Python dictionaries or lists.

```
import requests

url = "https://jsonplaceholder.typicode.com/users/1"
response = requests.get(url)

print("Status Code:", response.status_code)  # Output: 200
print("User Data:", response.json())         # Output: Python dictionary containing user data
```

(Sending a GET request and deserializing JSON response data)

3.3 Synchronous Nature of API Calls

Normal Python programs using `requests` execute **synchronously**.

- Code runs sequentially statement by statement.
- When `requests.get()` is invoked, Python halts and waits until the remote server replies before advancing to subsequent lines.
- If an unhandled network error occurs, execution terminates at that line.

3.4 Processing Multiple Records and Filtering Data

When querying a collection endpoint (e.g., `https://jsonplaceholder.typicode.com/users` without an ID), the API returns a **list of dictionaries**. Python iterates through this list using standard loops and conditional filtering:

```
import requests

url = "https://jsonplaceholder.typicode.com/users"
response = requests.get(url)

if response.status_code == 200:
    users = response.json()
    print("Total Users Retrieved:", len(users))

    # 1. Iterating through all users
    for user in users:
        print(f"ID: {user['id']} | Name: {user['name']} | City: {user['address']['city']}")

    # 2. Filtering users by city
    search_city = input("\nEnter city name to filter: ").strip()
    found = False
    for user in users:
        if user["address"]["city"].lower() == search_city.lower():
            print(f"Matched: {user['name']} ({user['email']})")
            found = True
    if not found:
        print("No user located in that city.")
else:
    print("Failed to fetch user collection. Status:", response.status_code)
```

(Processing and searching lists of user records returned from a collection endpoint)

---

4. Error Handling in API-Based Programs

4.1 Common Failure Points in Web APIs

Unlike local scripts, API-based software must withstand external disruptions:

- Missing internet connectivity
- Incorrect or malformed URLs
- Remote server downtime or maintenance
- Invalid or non-existent resource IDs (resulting in `404 Not Found`)
- Server-side processing crashes (resulting in `500 Internal Server Error`)
- Network latency and connection timeouts
- Mismatched or missing JSON keys causing local `KeyError` exceptions

---

4.2 Status Code Verification

The first line of defense is inspecting **`response.status_code`** before attempting to parse data:

- **`200 OK`**: Request succeeded; safe to parse JSON.
- **`404 Not Found`**: Target resource does not exist on the server.
- **`500 Internal Server Error`**: Server encountered an unhandled crash.

```
if response.status_code == 200:
    data = response.json()
    print("Name:", data["name"])
elif response.status_code == 404:
```

```
    print("User not found.")
else:
    print("API Error Code:", response.status_code)
```

(Evaluating HTTP status codes for conditional program execution)

4.3 Network Exceptions & Timeout Management

Network calls should specify a **`timeout`** argument to prevent the script from hanging indefinitely if a server becomes unresponsive. Network failures must be caught using **`try-except`** blocks with specialized exception classes from `requests.exceptions`:

Exception Class	Cause
`requests.exceptions.ConnectionError`	Physical connection failure, DNS resolution failure, or missing internet connection.
`requests.exceptions.Timeout`	Server failed to respond within the duration allocated by `timeout`.
`requests.exceptions.HTTPError`	Raised when calling `response.raise_for_status()` on 4xx or 5xx error codes.
`requests.exceptions.RequestException`	Base exception class catching all general `requests` library errors.

```
import requests

url = "https://jsonplaceholder.typicode.com/users/1"

try:
    # Set a 5-second timeout limit
    response = requests.get(url, timeout=5)
    response.raise_for_status()  # Raises HTTPError if status code is 4xx or 5xx

    data = response.json()
    print("User Name:", data["name"])

except requests.exceptions.ConnectionError:
    print("Network Error: Unable to connect. Check internet connection.")
except requests.exceptions.Timeout:
    print("Timeout Error: The remote server took too long to respond.")
except requests.exceptions.HTTPError as err:
    print(f"HTTP Error: {err.response.status_code}")
except requests.exceptions.RequestException as err:
    print("General Request Error:", err)
```

(Robust network error handling with timeouts and exception hierarchies)

---

5. Lab Exercises: Complete Implementations

Lab Exercise 4: Fetch Data from a Public API, Process JSON Response, and Display Meaningful Information

- **Objective**: Write an interactive Python application that accepts a `User ID` from the user, retrieves the record from the public JSONPlaceholder API (`https://jsonplaceholder.typicode.com/users/<id>`), extracts both direct fields and nested JSON objects (`address` and `company`), and handles numeric validation, invalid user IDs (`404`), and network dropouts gracefully.

```
"""
Lab Exercise 4: Fetching & Processing JSON Data from a Public REST API
Target Endpoint: JSONPlaceholder (https://jsonplaceholder.typicode.com/users)
Concepts Covered: requests.get, response.status_code, response.json(), nested JSON, exceptions
"""
import requests

def fetch_user_information():
    print("=" * 55)
    print("  PUBLIC REST API CONSUMER - USER INFORMATION FINDER")
    print("=" * 55)

    user_input = input("Enter User ID (1-10): ").strip()

    # 1. Input Validation: Check numeric conversion and range
    try:
        user_id = int(user_input)
        if user_id < 1 or user_id > 10:
            print("[!] Range Warning: Public testing endpoint provides IDs 1 through 10.")
    except ValueError:
        print("[!] Input Error: Please enter a valid numeric User ID.")
        return

    # 2. Dynamic Endpoint URL Construction
    url = f"https://jsonplaceholder.typicode.com/users/{user_id}"

    # 3. Request Transmission with Timeout & Exception Handling
    try:
        response = requests.get(url, timeout=5)

        # 4. Status Code Evaluation
        if response.status_code == 200:
            data = response.json()  # Convert JSON string to Python dictionary

            print("\n" + "-" * 25 + " USER PROFILE " + "-" * 25)
            print(f" User ID       : {data['id']}")
            print(f" Full Name     : {data['name']}")
            print(f" Username      : {data['username']}")
            print(f" Email Address : {data['email']}")
            print(f" Contact Phone : {data['phone']}")
```

```
            print(f" Website       : {data['website']}")

            # Extracting nested JSON values
            print(f" Residential City : {data['address']['city']}")
            print(f" Postal Zipcode   : {data['address']['zipcode']}")
            print(f" Organization     : {data['company']['name']}")
            print("-" * 64)

        elif response.status_code == 404:
            print(f"\n[!] Error 404: User record with ID '{user_id}' was not found on the server.")
        else:
            print(f"\n[!] API Error: Received unexpected HTTP status code {response.status_code}.")

    except requests.exceptions.ConnectionError:
        print("\n[!] Connection Error: Could not connect to the remote server.")
        print("    Please check your network connection.")
    except requests.exceptions.Timeout:
        print("\n[!] Timeout Error: Server response exceeded the 5-second threshold.")
    except requests.exceptions.RequestException as err:
        print(f"\n[!] Network Exception Occurred: {err}")

if __name__ == "__main__":
    fetch_user_information()
```

(Complete interactive public API consumer with nested JSON extraction and error handling)

Lab Exercise 5: Organize Files in Folders Based on File Type and Log the Operations Performed

- **Objective**: Write a Python automation utility that scans a designated folder, extracts file extensions, creates categorized subdirectories automatically, moves files into their corresponding category folder, handles unrecognized file formats by moving them into an `Others` folder, tracks the total count of organized files, and logs every move operation with timestamps into a log file using Python's **`logging`** module.

Modules Required:

- **`os`**: Navigating paths, listing folder contents (`os.listdir`), verifying file status (`os.path.isfile`), joining path strings (`os.path.join`), creating directories (`os.makedirs`), and splitting filename from extension (`os.path.splitext`).
- **`shutil`**: Moving files across directories (`shutil.move`).
- **`logging`**: Recording an audit trail of operations in `organizer.log`.

```
"""
Lab Exercise 5: File Organizer by Extension with Operations Logging
Modules Used: os, shutil, logging
Concepts: Path manipulation, Directory creation, File moving, Audit logging
"""
import os
import shutil
import logging

def organize_directory(source_folder="TestFiles"):
    # 1. Setup Audit Logging Configuration
    log_file_path = "organizer.log"
    logging.basicConfig(
        filename=log_file_path,
        level=logging.INFO,
        format="%(asctime)s - %(levelname)s - %(message)s"
    )

    print("=" * 60)
    print("      AUTOMATED FILE ORGANIZER AND AUDIT LOGGER")
    print("=" * 60)

    # Verify source directory existence
    if not os.path.exists(source_folder):
        os.makedirs(source_folder, exist_ok=True)
        print(f"Created sample folder '{source_folder}'. Place files inside and run again.")
        return

    # 2. Extension-to-Folder Mapping Dictionary
    folder_mapping = {
        ".pdf": "PDFs",
        ".png": "Images",
        ".jpg": "Images",
        ".jpeg": "Images",
        ".docx": "Documents",
        ".txt": "Documents",
        ".py": "Python_Files",
        ".csv": "Data_Files"
    }

    organized_count = 0

    # 3. Read All Entries from the Source Directory
    entries = os.listdir(source_folder)

    for entry in entries:
        file_path = os.path.join(source_folder, entry)

        # Ensure we only process files, skipping already existing folders
        if os.path.isfile(file_path):
            # Exclude the active log file if placed inside the same folder
            if entry == log_file_path:
                continue
```

```
            # Separate base name and extension
            name, extension = os.path.splitext(entry)
            extension = extension.lower()

            # Determine destination folder name
            if extension in folder_mapping:
                destination_subfolder = folder_mapping[extension]
            else:
                destination_subfolder = "Others"  # Fallback for unrecognized file types

            # Create destination directory if it does not already exist
            destination_dir = os.path.join(source_folder, destination_subfolder)
            os.makedirs(destination_dir, exist_ok=True)

            # Define full target destination path
            destination_path = os.path.join(destination_dir, entry)

            # Move the file
            shutil.move(file_path, destination_path)
            organized_count += 1

            # Console notification & persistent audit logging
            print(f"[✓] Moved: {entry:<20} ──> {destination_subfolder}/")
            logging.info("Moved '%s' to folder '%s'", entry, destination_subfolder)

    print("-" * 60)
    print(f"Execution Summary: Total files organized = {organized_count}")
    print(f"Operation audit trail successfully recorded in '{log_file_path}'.")
    print("=" * 60)

if __name__ == "__main__":
    organize_directory("TestFiles")
```

(Complete file classification script with extension extraction, directory creation, and logging)

Sample Audit Trail (`organizer.log` output):

```
2026-09-06 14:20:10,102 - INFO - Moved 'report.pdf' to folder 'PDFs'
2026-09-06 14:20:10,105 - INFO - Moved 'image.jpg' to folder 'Images'
2026-09-06 14:20:10,108 - INFO - Moved 'notes.txt' to folder 'Documents'
2026-09-06 14:20:10,111 - INFO - Moved 'program.py' to folder 'Python_Files'
2026-09-06 14:20:10,114 - INFO - Moved 'students.csv' to folder 'Data_Files'
2026-09-06 14:20:10,117 - INFO - Moved 'archive.zip' to folder 'Others'
```

(Audit trail generated by the logging module)

---

6. Summary Comparison: API Error Handling Strategies

Strategy	When to Use	Key Syntax / Function
Status Code Checking	Inspecting expected server outcomes (`200`, `404`, `500`).	`if response.status_code == 200:`
HTTP Error Raising	Automatically raising exceptions for `4xx` client errors or `5xx` server failures.	`response.raise_for_status()`
Timeout Protection	Preventing slow or frozen connections from hanging scripts.	`requests.get(url, timeout=5)`
Network `try-except`	Catching physical network drops, DNS errors, or timeouts gracefully.	`except requests.RequestException:`
Data Key Checking	Guarding against missing dictionary attributes inside dynamic JSON payloads.	`user.get("key")` or `try...except KeyError:`

Unit 3: Introduction to Web Frameworks

---

1. Web Frameworks & Streamlit Introduction

1.1 What is a Web Framework?

A **web framework** provides ready-made tools, architectural patterns, and libraries that help developers construct web applications without having to implement low-level networking, socket communication, or routing protocols from scratch.

In a conventional Python script, user interaction occurs via the terminal or console using standard input/output functions (`input()` and `print()`). A web framework bridges this gap by rendering inputs and outputs as an **interactive graphical web interface** accessible inside any modern web browser.

```
Normal Python    ──>   Terminal / Console Output
Python + Web Framework  ──>   Interactive Web Browser Interface
```

(Progression from console execution to web interfaces)

Prominent Python web frameworks include **Flask**, **Django**, **FastAPI**, and **Streamlit**. For this curriculum, the core focus centers on **Streamlit** (for data dashboards and rapid GUI-like web tools) and **Flask** (for route-driven web services and custom web applications).

1.2 Introduction to Streamlit

Streamlit is an open-source Python framework designed for quickly building interactive web applications, data dashboards, and machine learning/AI prediction interfaces without requiring frontend HTML, CSS, or JavaScript coding.

Primary Use Cases:

- Data analytics dashboards and business intelligence tools
- Student examination and academic performance dashboards
- Machine learning model prediction and parameter-tuning interfaces
- Survey and feedback analysis systems
- Interactive sales, financial, and data visualization tools

1.3 Streamlit Setup and Environment Verification

Before building Streamlit applications, verify the Python installation and install the required packages using the terminal:

```
# Verify Python installation
python --version

# Install Streamlit along with supporting analytics and visualization libraries
pip install streamlit pandas matplotlib plotly

# Verify successful Streamlit installation
streamlit --version
```

(Terminal commands for installing and verifying Streamlit and data analysis dependencies)

1.4 Creating and Executing Your First Streamlit Application

A Streamlit script is a standard Python file (e.g., `first_app.py`). Rather than running it with `python first_app.py`, it is launched using the dedicated `streamlit run` command:

```
# first_app.py
import streamlit as st

# Display main title
st.title("My First Streamlit App")

# Display general text output
st.write("Welcome to Streamlit!")
st.write("This webpage is created using Python.")
```

(Introductory Streamlit script)

To execute the application, run the following in your terminal:

```
streamlit run first_app.py
```

(Command to launch the local Streamlit web server)

Streamlit spins up a local web server and automatically opens your default web browser displaying the application at `http://localhost:8501`.

---

2. Building Streamlit Applications: Displaying Content & Interactive Components

2.1 Text, Headings, and Status Indicators

Streamlit includes high-level methods to structure textual content and display operational status alerts:

Streamlit Function	Visual Behavior / Purpose	Example Syntax
`st.title()`	Renders the primary top-level heading	`st.title("Student Portal")`
`st.header()`	Renders a primary section header	`st.header("Academic Details")`
`st.subheader()`	Renders a secondary subsection header	`st.subheader("Marks Summary")`
`st.write()`	General display method for strings, numbers, DataFrames, and charts	`st.write("Hello World")`
`st.caption()`	Renders smaller muted explanatory text	`st.caption("Updated today")`
`st.markdown()`	Renders custom Markdown-formatted text	`st.markdown("### Heading")`
`st.success()`	Renders a green success alert banner	`st.success("Result: PASS")`
`st.error()`	Renders a red error alert banner	`st.error("Result: FAIL")`
`st.warning()`	Renders a yellow cautionary alert banner	`st.warning("Underweight")`
`st.info()`	Renders a blue informational alert banner	`st.info("Upload CSV")`

```
import streamlit as st

st.title("University Academic Portal")
st.header("Department of Computer Science")
st.subheader("Advanced Python Programming")
st.write("Welcome to the interactive student management dashboard.")
st.success("Application loaded successfully!")
```

(Structuring typography and alerts in Streamlit)

2.2 Interactive Input Widgets

Streamlit widgets bind directly to Python variables. Whenever a user interacts with a widget (typing text, toggling a checkbox, selecting a dropdown), Streamlit automatically reruns the Python script from top to bottom with the updated variable value.

```
import streamlit as st

st.title("Student Input Widget Demo")

# 1. Single-line text input
name = st.text_input("Enter Student Name")

# 2. Number input with bounded limits
marks = st.number_input("Enter Marks", min_value=0, max_value=100, value=50)

# 3. Numeric slider for ranges
attendance = st.slider("Select Attendance (%)", min_value=0, max_value=100, value=75)

# 4. Dropdown select box
course = st.selectbox("Select Course", ["BCA", "B.Sc. CS", "MCA", "M.Sc. DS"])

# 5. Radio button group for single choice
year = st.radio("Select Year", ["First Year", "Second Year", "Third Year"])

# 6. Checkbox for boolean confirmation
agree = st.checkbox("I verify that the above information is accurate")

# 7. Action Button
if st.button("Submit Profile"):
    if agree:
        st.success(f"Profile saved for {name} ({course}, {year}). Marks: {marks}, Attendance: {attendance}%.")
    else:
        st.warning("Please check the confirmation box before submitting.")
```

(Interactive input components available in Streamlit)

2.3 Dashboard Layouts, Metrics, and Visualizations

Streamlit simplifies dashboard layout through **columns**, **metric cards**, and **built-in charts** integrated directly with **Pandas DataFrames**:

- **`st.columns(n)`**: Splits the browser window horizontally into `n` responsive side-by-side columns.
- **`col.metric(label, value)`**: Displays large key performance indicators (KPIs) in dedicated summary cards.
- **`st.dataframe(df)`**: Renders a dynamic, sortable, and scrollable tabular spreadsheet.
- **`st.bar_chart(df)`**: Automatically visualizes numeric DataFrame columns as a bar chart without requiring low-level plotting code.

```
import streamlit as st
import pandas as pd

st.title("Class Performance Overview")

# Tabular data represented as a dictionary and converted to a Pandas DataFrame
data = {
    "Student": ["Amit", "Priya", "Rahul", "Sneha", "Neha"],
    "Marks":
}
df = pd.DataFrame(data)

# Dashboard KPI metric cards across 3 columns
col1, col2, col3 = st.columns(3)
col1.metric("Total Students", len(df))
col2.metric("Class Average", f"{df['Marks'].mean():.2f}")
col3.metric("Highest Marks", df["Marks"].max())

# Display interactive data table
st.subheader("Student Data Table")
st.dataframe(df, use_container_width=True)

# Display native bar chart indexed by student name
st.subheader("Performance Comparison")
st.bar_chart(df.set_index("Student"))
```

(Creating a dashboard layout with summary metrics, tables, and bar charts)

---

3. Deploying Streamlit Applications Online

3.1 Localhost vs. Cloud Production Architecture

During local development, running `streamlit run app.py` starts a local development server (`http://localhost:8501`) that is accessible only on your local machine.

Deployment is the process of hosting the application on a public cloud server so that any user across the internet can access it via a dedicated URL (e.g., `https://<app-name>.streamlit.app`).

```
Local Development:
Your Computer  ──>  localhost:8501  ──>  Accessible only by you

Cloud Deployment:
Internet User  ──>  https://app-name.streamlit.app  ──>  Accessible globally
```

(Local development versus deployed public web applications)

3.2 The Role of `requirements.txt`

Your local computer already has libraries like `streamlit`, `pandas`, and `matplotlib` installed in its Python environment. However, when deploying code to an isolated cloud container, the server must be informed which external dependencies must be installed to run the application.

This specification is handled by a plain text file named **`requirements.txt`** placed in the root directory alongside your main Python file:

```
streamlit
pandas
matplotlib
plotly
```

(Standard requirements.txt declaring external dependencies for deployment)

3.3 End-to-End Streamlit Cloud Deployment Workflow

Streamlit Community Cloud (`share.streamlit.io`) deploys directly from GitHub repositories:

```
Step 1: Write Python Application (app.py)
        |
        ▼
Step 2: Declare Dependencies (requirements.txt)
        |
        ▼
Step 3: Test Locally (streamlit run app.py)
        |
        ▼
Step 4: Push Files to a GitHub Repository
        |
        ▼
Step 5: Sign in to Streamlit Community Cloud (share.streamlit.io) via GitHub
        |
        ▼
Step 6: Select Repository, Branch (main), and Main file path (app.py)
        |
        ▼
Step 7: Click Deploy
        |
        ▼
Public Web Application URL Generated (https://<app-name>.streamlit.app)
```

(End-to-end workflow for deploying Streamlit applications to the cloud)

Behind the Scenes During Deployment:

1. Streamlit Community Cloud clones the GitHub repository.
2. The platform reads `requirements.txt` and installs all listed libraries into an isolated container using `pip`.
3. The platform executes `streamlit run app.py`.
4. A public URL on the `*.streamlit.app` subdomain is assigned to your live web app.

---

4. Introduction to Flask

4.1 Overview of Flask

Flask is a lightweight, flexible web framework (often called a **microframework**) for Python. Unlike heavyweight frameworks, Flask does not enforce a rigid project structure, providing core essentials such as request handling, URL routing, and template rendering while leaving developers free to integrate any database or library.

Comparison: Streamlit vs. Flask:

Feature	Streamlit	Flask
Primary Focus	Data dashboards, internal tools, ML interfaces	Custom web portals, REST APIs, multi-page websites
Frontend Coding	Pure Python; zero HTML/CSS/JS required	Python backend paired with HTML templates
Execution Model	Script re-executes on user interaction	Event/request-driven via explicit route handlers
URL Routing	Single-page by default (or multi-page folder)	Custom URL routes using `@app.route()`

4.2 Application Initialization & URL Routing

A Flask web application is initialized by instantiating the `Flask` class. **Routing** maps a specific Uniform Resource Locator (URL) path entered into a browser to a corresponding Python function (known as a **view function**) using the `@app.route()` decorator.

```python
from flask import Flask

# Initialize the Flask WSGI application
app = Flask(__name__)

# Route decorator binding URL path '/' to home()
@app.route("/")
def home():
    return "Welcome to the Student Portal Home Page!"

# Route decorator binding URL path '/about' to about()
@app.route("/about")
```

```
def about():
    return "This is the About Page of our institution."

# Route decorator binding URL path '/courses' to courses()
@app.route("/courses")
def courses():
    return "Available Courses: BCA, B.Sc. Data Science, MCA, M.Sc."

if __name__ == "__main__":
    # Start the Flask development server on localhost:5000
    app.run(debug=True)
```

(Multi-route Flask application with route decorators and view functions)

4.3 Handling HTTP Methods (`GET` & `POST`) and Form Data

Web forms submit data using the HTTP **`POST`** method, while standard page navigation uses the **`GET`** method:

- **`methods=["GET", "POST"]`**: Allows a route to accept both form rendering (`GET`) and form submission (`POST`).
- **`request.form["field_name"]`**: Extracts submitted input field values from incoming POST payloads.
- **`redirect(url_for("view_function_name"))`**: Safely redirects the browser to another route after processing form data.
- **`render_template_string()`** or **`render_template()`**: Merges dynamic Python data with HTML templates.

---

5. Complete Implementations for Unit-3 Lab Exercises

Lab Exercise 6: Building and Deploying a Streamlit Application

- **Objective**: Build a complete, production-grade **Student Performance Dashboard** in Streamlit that supports CSV dataset uploads, calculates student averages and pass/fail status across subjects, shows KPI metrics, provides a student selector with subject-level breakdown bar charts, includes dynamic score filtering via sliders, and packages the app for cloud deployment with a `requirements.txt` file.

1. Application Code (`app.py`):

```
"""
Lab Exercise 6: Building and Deploying a Streamlit Application
Title: Student Performance Analyzer & Dashboard
Features: CSV Upload, Pandas Processing, Metrics, Charts, and Filtering
"""
import streamlit as st
import pandas as pd
import matplotlib.pyplot as plt

# ------------------------------------------------------------
# Page Configuration & Header
# ------------------------------------------------------------
st.set_page_config(page_title="Student Performance Analyzer", layout="wide")
st.title("🎓 Student Performance Analyzer Dashboard")
st.caption("Upload a student dataset in CSV format to analyze academic performance interactively.")

# ------------------------------------------------------------
# Sidebar: Dataset Upload and Settings
# ------------------------------------------------------------
with st.sidebar:
    st.header("Upload Dataset")
    uploaded_file = st.file_uploader("Upload CSV File", type=["csv"])
    st.divider()
    passing_marks = st.slider("Passing Marks Threshold", min_value=0, max_value=100, value=40)
    st.info("Ensure the first column is 'Name' followed by numeric subject scores.")

# ------------------------------------------------------------
# Main Processing Logic
# ------------------------------------------------------------
if uploaded_file is not None:
    # 1. Read CSV using Pandas
    df = pd.read_csv(uploaded_file)
    st.success("CSV file uploaded and parsed successfully!")

    # Verify student name column and numeric subject columns
    student_col = df.columns
    subject_cols = df.select_dtypes(include="number").columns.tolist()

    if not subject_cols:
        st.error("No numeric subject columns detected in the uploaded CSV.")
        st.stop()

    # 2. DataFrame Calculations: Average and Status
    df["Average"] = df[subject_cols].mean(axis=1)
    df["Result"] = df[subject_cols].apply(
        lambda row: "PASS" if all(row >= passing_marks) else "FAIL", axis=1
    )

    # ------------------------------------------------------------
    # Overview Metrics (Columns)
    # ------------------------------------------------------------
    st.subheader("Class Overview")
    col1, col2, col3, col4, col5 = st.columns(5)
    col1.metric("Total Students", len(df))
    col2.metric("Subjects", len(subject_cols))
    col3.metric("Class Average", f"{df['Average'].mean():.2f}")
```

```
    col4.metric("Passed", int((df["Result"] == "PASS").sum()))
    col5.metric("Failed", int((df["Result"] == "FAIL").sum()))

    st.divider()

    # ------------------------------------------------------
    # Tab 1: Dataset & Tab 2: Individual Analysis & Tab 3: Comparisons
    # ------------------------------------------------------
    tab1, tab2, tab3 = st.tabs(["🔲 Complete Dataset", "👤 Individual Student Analysis", "📊 Class Comparison"])

    with tab1:
        st.subheader("Student Dataset")
        st.dataframe(df, use_container_width=True)

        # Dynamic filtering using slider
        st.subheader("Filter Students by Minimum Average")
        min_avg = st.slider("Select Minimum Average", 0, 100, 50)
        filtered_df = df[df["Average"] >= min_avg]
        st.write(f"Students with Average ≥ {min_avg}: **{len(filtered_df)} found**")
        st.dataframe(filtered_df, use_container_width=True)

    with tab2:
        st.subheader("Select Student for Detailed Analysis")
        selected_student = st.selectbox("Choose a Student", df[student_col].tolist())
        student_data = df[df[student_col] == selected_student].iloc

        m_col1, m_col2, m_col3 = st.columns(3)
        m_col1.metric("Average Score", f"{student_data['Average']:.2f}")
        m_col2.metric("Result Status", student_data["Result"])
        m_col3.metric("Total Marks", f"{student_data[subject_cols].sum():.0f}")

        # Display performance bar chart for selected student
        st.write(f"### {selected_student}'s Subject Performance")
        student_marks_df = pd.DataFrame({
            "Subject": subject_cols,
            "Marks": [student_data[s] for s in subject_cols]
        })
        st.bar_chart(student_marks_df.set_index("Subject"))

    with tab3:
        st.subheader("Class Performance Charts")
        # Comparison of all student averages
        avg_chart_df = df[[student_col, "Average"]].set_index(student_col)
        st.bar_chart(avg_chart_df)

        # Subject-wise average breakdown
        st.write("### Subject-wise Class Averages")
        subj_avg_df = pd.DataFrame({
            "Subject": subject_cols,
            "Class Average": [df[s].mean() for s in subject_cols]
        }).set_index("Subject")
        st.bar_chart(subj_avg_df)

else:
    st.info("👈 Please upload a student marks CSV file from the sidebar to begin.")
    st.markdown("""
    #### Sample CSV Format:
    ```csv
    Name,Python,Java,DBMS,AI
    Amit,78,65,82,90
    Priya,92,88,90,95
    Rahul,67,75,70,68
    Sneha,85,91,89,92
    ```
    """)
```

(Complete student performance analyzer with file upload, data processing, metrics, tabs, and charts)

2. Dependency Specification (`requirements.txt`):

```
streamlit
pandas
matplotlib
```

(Required packages text file for cloud container building)

3. Step-by-Step Deployment Guide:

1. **Local Directory Setup**: Save `app.py` and `requirements.txt` inside a local project folder (e.g., `StreamlitDemo/`).
2. **Local Validation**: Run `streamlit run app.py` in your terminal to verify that the app loads and functions properly at `http://localhost:8501`.
3. **Push to GitHub**: Initialize a Git repository, commit both files, and push them to a public GitHub repository (e.g., `https://github.com/your-username/student-performance-dashboard`).
4. **Deploy via Streamlit Cloud**:
   - Navigate to share.streamlit.io and authenticate using your GitHub account.
   - Click **Create app** and select **I already have an app**.
   - Select your repository (`student-performance-dashboard`), branch (`main`), and set the main file path to `app.py`.
   - Click **Deploy!**
5. **Access Live Application**: Streamlit Cloud installs the dependencies listed in `requirements.txt` and publishes your app to a live URL (e.g., `https://student-performance-dashboard.streamlit.app`).

Lab Exercise 7: Building and Deploying a Flask Application

- **Objective**: Construct a multi-route Flask web application for a **Student Registration Portal** supporting homepage routing, an interactive HTML registration form handling `GET` and `POST` requests, redirect navigation, a live student records table, and deployment configuration.

Application Code (`app.py`):

```
"""
Lab Exercise 7: Building and Deploying a Flask Application
Title: Multi-Route Student Registration Portal
Features: URL Routing, Form Handling (GET & POST), Data Storage, Dynamic HTML Rendering
"""
from flask import Flask, render_template_string, request, redirect, url_for

app = Flask(__name__)

# In-memory storage list for registered student records
students_database = [
    {"name": "Amit Sharma", "email": "amit@example.com", "course": "BCA"},
    {"name": "Priya Patel", "email": "priya@example.com", "course": "B.Sc. Data Science"}
]

# ------------------------------------------------------------
# Embedded HTML Templates
# ------------------------------------------------------------
BASE_NAV = """
<nav style="background-color: #333; padding: 12px; margin-bottom: 20px;">
    <a href="/" style="color: white; margin-right: 20px; text-decoration: none; font-weight: bold;">Home</a>
    <a href="/register" style="color: white; margin-right: 20px; text-decoration: none; font-weight: bold;">Register Student</a>
    <a href="/students" style="color: white; margin-right: 20px; text-decoration: none; font-weight: bold;">View Students</a>
    <a href="/about" style="color: white; text-decoration: none; font-weight: bold;">About</a>
</nav>
"""

HOME_HTML = BASE_NAV + """
<div style="font-family: Arial; padding: 20px; max-width: 800px; margin: auto;">
    <h1>🎓 Welcome to the University Student Portal</h1>
    <p>This web application is built using the <b>Flask microframework</b> in Python.</p>
    <p>Use the navigation menu above to register new students or review existing enrollment records.</p>
</div>
"""

REGISTER_HTML = BASE_NAV + """
<div style="font-family: Arial; padding: 20px; max-width: 500px; margin: auto; border: 1px solid #ddd; border-radius: 8px;">
    <h2>Student Registration Form</h2>
    <form method="POST" action="/register">
        <label><b>Full Name:</b></label><br>
        <input type="text" name="name" required style="width: 100%; padding: 8px; margin: 8px 0;"><br>

        <label><b>Email Address:</b></label><br>
        <input type="email" name="email" required style="width: 100%; padding: 8px; margin: 8px 0;"><br>

        <label><b>Programme / Course:</b></label><br>
        <select name="course" style="width: 100%; padding: 8px; margin: 8px 0;">
            <option value="BCA">BCA</option>
            <option value="B.Sc. Data Science">B.Sc. Data Science</option>
            <option value="MCA">MCA</option>
            <option value="M.Sc. AI">M.Sc. AI</option>
        </select><br><br>

        <input type="submit" value="Register Student" style="background-color: #28a745; color: white; border: none; padding: 10px 20px; curso
    </form>
</div>
"""

STUDENTS_HTML = BASE_NAV + """
<div style="font-family: Arial; padding: 20px; max-width: 800px; margin: auto;">
    <h2>Enrolled Students Directory</h2>
    <table border="1" cellpadding="10" cellspacing="0" style="width: 100%; border-collapse: collapse;">
        <thead style="background-color: #f2f2f2;">
            <tr>
                <th>#</th>
                <th>Student Name</th>
                <th>Email Address</th>
                <th>Enrolled Course</th>
            </tr>
        </thead>
        <tbody>
            {% for s in students %}
            <tr>
                <td>{{ loop.index }}</td>
                <td>{{ s.name }}</td>
                <td>{{ s.email }}</td>
                <td>{{ s.course }}</td>
            </tr>
            {% endfor %}
        </tbody>
    </table>
    <br>
    <a href="/register">+ Register Another Student</a>
</div>
"""

ABOUT_HTML = BASE_NAV + """
<div style="font-family: Arial; padding: 20px; max-width: 800px; margin: auto;">
```

```
    <h2>About This Web Application</h2>
    <p><b>Unit 3 Practical Demonstration:</b> Python Web Frameworks with Flask.</p>
    <p>Demonstrates WSGI routing, HTTP GET/POST handling, and dynamic template rendering.</p>
</div>
"""

# ------------------------------------------------------------
# Flask URL Route Handlers
# ------------------------------------------------------------
@app.route("/")
def home():
    """Renders the Portal Home Page."""
    return render_template_string(HOME_HTML)

@app.route("/register", methods=["GET", "POST"])
def register():
    """Handles both GET (form display) and POST (submission processing)."""
    if request.method == "POST":
        # Extract form field values from request.form dictionary
        new_student = {
            "name": request.form["name"].strip(),
            "email": request.form["email"].strip(),
            "course": request.form["course"]
        }
        students_database.append(new_student)
        # Redirect to the student listing page after successful submission
        return redirect(url_for("view_students"))

    # If request is GET, render the registration form
    return render_template_string(REGISTER_HTML)

@app.route("/students")
def view_students():
    """Displays all registered student records dynamically."""
    return render_template_string(STUDENTS_HTML, students=students_database)

@app.route("/about")
def about():
    """Renders the About Page."""
    return render_template_string(ABOUT_HTML)

if __name__ == "__main__":
    # Run development server on port 5001 with debug mode enabled
    app.run(debug=True, port=5001)
```

(Complete multi-route Flask web application handling GET/POST requests and rendering HTML templates)

Deployment Instructions for Flask:

1. **Requirements File (`requirements.txt`)**:

   ```
   Flask
   gunicorn
   ```

2. **WSGI Server**: In production environments (such as Render, Railway, or AWS), Flask applications are served using a production-grade WSGI HTTP server like **Gunicorn**:

   ```
   gunicorn app:app
   ```

3. **Execution**: Locally, execute `python app.py` and open your web browser to `http://localhost:5001`.

---

6. Summary Comparison: Streamlit vs. Flask

Evaluation Criterion	Streamlit Framework	Flask Microframework
Primary Philosophy	Turn data scripts into shareable web apps instantly	Lightweight, unopinionated web service foundation
Target Applications	Dashboards, data analysis, ML demos, BI utilities	Enterprise web portals, RESTful APIs, traditional websites
Component Syntax	High-level widgets (`st.selectbox`, `st.slider`)	HTML forms submitted to routes via `request.form`
Data Visualization	One-line charts (`st.bar_chart`, Plotly, Altair)	Requires embedding chart images or JavaScript chart libraries
Execution Loop	Entire script executes sequentially on every interaction	Individual route view functions execute per HTTP request
Deployment Target	Streamlit Community Cloud (via GitHub)	Cloud platforms (Render, Heroku, AWS) via Gunicorn/WSGI